

Exploring Model Context Protocol (MCP) With Spring AI

Last updated: December 17, 2025



Written by: Hardik Singh Behl (<https://www.baeldung.com/author/hardiksinghbehl>)



Reviewed by: Liam Williams (<https://www.baeldung.com/editor/liamwilliams>)

Artificial Intelligence (<https://www.baeldung.com/category/artificial-intelligence>)

Spring AI (<https://www.baeldung.com/category/spring/spring-ai>)

LLM (<https://www.baeldung.com/tag/llm>) **MCP** (<https://www.baeldung.com/tag/mcp>)



Explore Spring Boot 3 and Spring 6 in-depth through building a full *REST API* with the framework:

>> The New "REST With Spring Boot" (/Course-RwS-NPI-EA-1-h4V3y)

1. Overview

Modern web applications are increasingly integrating with Large Language Models (LLMs) (/cs/large-language-models) to build solutions, which are not just limited to general knowledge-based question-answering.

To enhance an AI model's response and make it more context-aware, we can connect it to external sources like search engines, databases, and file systems. However, integrating and managing multiple data sources with different formats and protocols is a challenge.

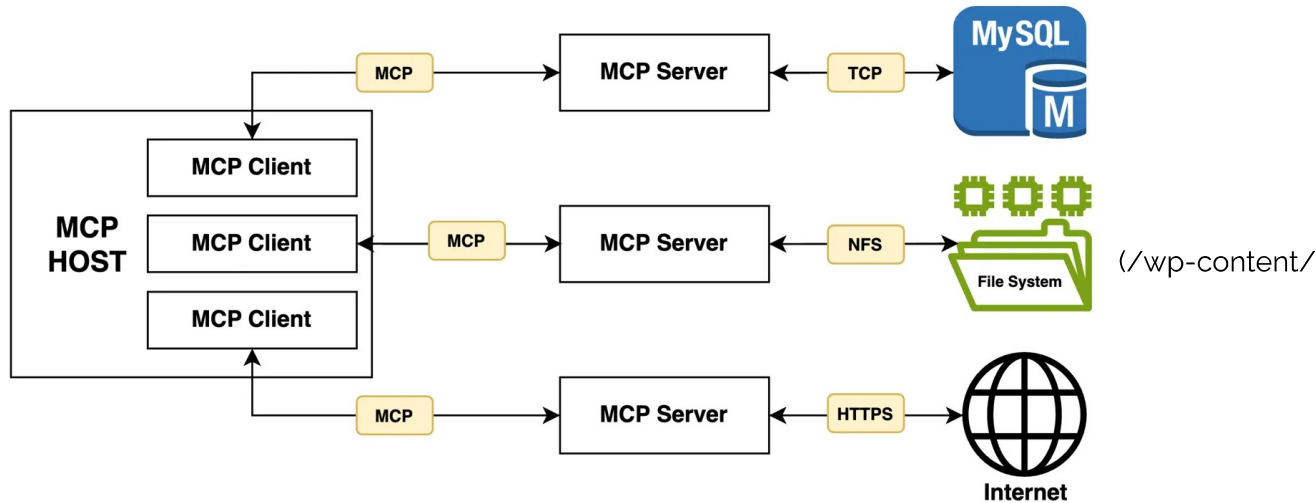
The Model Context Protocol (MCP), introduced by Anthropic (<https://www.anthropic.com/>), addresses this integration challenge and provides a standardized way to connect AI-powered applications with external data sources. Through MCP, we can build complex agents and workflows on top of a native LLM.

In this tutorial, we'll understand the concept of MCP by practically implementing its client-server

architecture using Spring AI. We'll create a simple chatbot and extend its capabilities through MCP servers to perform web searches, execute filesystem operations, and access custom business logic.

2. Model Context Protocol 101

Before we dive into the implementation, let's take a closer look at MCP and its various components:



uploads/2025/02/img_67bf6c9331f87.)

MCP follows a client-server architecture that revolves around several key components:

- **MCP Host:** is our main application that integrates with an LLM and requires it to connect with external data sources
- **MCP Clients:** are components that establish and maintain 1:1 connections with MCP servers
- **MCP Servers:** are components that integrate with external data sources and expose functionalities to interact with them
- **Tools:** refer to the executable functions/methods that the MCP servers expose for clients to invoke

Additionally, to handle communication between clients and servers, MCP provides two transport channels.

To enable communication through standard input and output streams with local processes and command-line tools, it provides the Standard Input/Output (stdio) transport type. Alternatively, for HTTP-based communication between clients and servers, it provides the Server-Sent Events (SSE) transport type.

MCP is a complex and vast topic, refer to the official documentation (<https://modelcontextprotocol.io/introduction>) to learn more.

3. Creating an MCP Host

Now that we've got a high-level understanding of MCP, let's start implementing the MCP architecture practically.

We'll be building a chatbot using Anthropic's Claude (/spring-ai-anthropics-claude-models) model, which will act as our MCP host. Alternatively, we can use a local LLM via Hugging Face or Ollama (/spring-ai-ollama-hugging-face-models), as the specific AI model is irrelevant for this demonstration.

3.1. Dependencies (//)

Let's start by adding the necessary dependencies to our project's *pom.xml* file:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-anthropic</artifactId>
  <version>1.0.1</version>
</dependency>
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-client</artifactId>
  <version>1.0.1</version>
</dependency>
```

The Anthropic starter dependency (<https://mvnrepository.com/artifact/org.springframework.ai/spring-ai-starter-model-anthropic>) is a wrapper around the Anthropic Message API (<https://docs.anthropic.com/en/api/messages>), and we'll use it to interact with the Claude model in our application.

Additionally, we import the MCP client starter dependency (<https://mvnrepository.com/artifact/org.springframework.ai/spring-ai-starter-mcp-client>), **which will allow us to configure clients inside our Spring Boot application that maintain 1:1 connections with the MCP servers.**

Given that we're using multiple Spring AI starters in our project, let's also include the Spring AI Bill of Materials (BOM) (<https://mvnrepository.com/artifact/org.springframework.ai/spring-ai-bom>) in our *pom.xml*:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>1.0.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

With this addition, we can now remove the *version* tag from both of our starter dependencies. **The BOM eliminates the risk of version conflicts and ensures our Spring AI dependencies are compatible with each other.**

Next, let's configure our Anthropic API key (<https://console.anthropic.com/settings/keys>) and chat model in the *application.yaml* file:

```
spring:
  ai:
    anthropic:
      api-key: ${ANTHROPIC_API_KEY}
      chat:
        options:
          model: claude-opus-4-20250514
```

We use the *\$\$* property placeholder to load the value of our API Key from an environment variable (*/spring-boot-properties-env-variables#use-environment-variable-in-applicationyaml-file*).

Additionally, we specify Claude 4 Opus (<https://www.anthropic.com/claude/opus>) by Anthropic, using the *claude-opus-4-20250514* model ID. Feel free to explore and use a different model (<https://docs.anthropic.com/en/docs/about-claude/models#model-names>) based on requirements.

On configuring the above properties, **Spring AI automatically creates a bean of type *ChatModel*, allowing us to interact with the specified model.**

3.2. Configuring MCP Clients for Brave Search and Filesystem Servers

Now, **let's configure MCP clients for two pre-built MCP server implementations: Brave Search (<https://github.com/modelcontextprotocol/servers-archived/tree/main/src/brave-search>) and Filesystem (<https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem>)**. These servers will enable our chatbot to perform web searches and filesystem operations.

Let's start by registering an MCP client for the Brave search MCP server in the *application.yaml* file:

```
spring:
  ai:
    mcp:
      client:
        stdio:
          connections:
            brave-search:
              command: npx
              args:
                - "-y"
                - "@modelcontextprotocol/server-brave-search"
          env:
            BRAVE_API_KEY: ${BRAVE_API_KEY}
```

Here, we configure a *client* with *stdio* transport. **We specify the *npx* (<https://docs.npmjs.com/cli/v11/commands/npx>) *command* to download and run the TypeScript-based *@modelcontextprotocol/server-brave-search* (<https://www.npmjs.com/package/@modelcontextprotocol/server-brave-search>) package and use the *-y* flag to confirm all the installation prompts.**

Additionally, we provide the *BRAVE_API_KEY* (<https://api-dashboard.search.brave.com/app/keys>) as an environment variable.

Next, let's configure an MCP client for the Filesystem MCP server:

```
spring:
  ai:
    mcp:
      client:
        stdio:
          connections:
            filesystem:
              command: npx
              args:
                - "-y"
                - "@modelcontextprotocol/server-filesystem"
                - "./"
```

Similar to the previous configuration, we specify the *command* and arguments required to run the Filesystem MCP server package (<https://www.npmjs.com/package/@modelcontextprotocol/server-filesystem>). This setup allows our chatbot to perform operations like creating, reading, and writing files in the specified directory.

Here, we only configure the current directory (*./*) to be used for filesystem operations, however, we can specify multiple directories by adding them to the *args* list.

During application startup, Spring AI will scan our configurations, create the MCP clients, and establish connections with their corresponding MCP servers. **It also creates a bean of type *SyncMcpToolCallbackProvider*, which provides a list of all the tools exposed by the configured MCP**

servers.

(/)

3.3. Building a Basic Chatbot

With our AI model and MCP clients configured, let's build a simple chatbot:

```
@Bean
ChatClient chatClient(ChatModel chatModel, SyncMcpToolCallbackProvider toolCallbackProvider) {
    return ChatClient
        .builder(chatModel)
        .defaultToolCallbacks(toolCallbackProvider.getToolCallbacks())
        .build();
}
```

We start by creating a bean of type *ChatClient* using the *ChatModel* and *SyncMcpToolCallbackProvider* beans. **The *ChatClient* class will act as our main entry point for interacting with our chat completion model, i.e., Claude 4 Opus.**

Next, let's inject the *ChatClient* bean to create a new *ChatbotService* class:

```
String chat(String question) {
    return chatClient
        .prompt()
        .user(question)
        .call()
        .content();
}
```

We create a *chat()* method where we pass the user's *question* to the *chat client* bean and simply return the AI model's response.

Now that we've implemented our service layer, **let's expose a REST API on top of it:**

```
@PostMapping("/chat")
ResponseBody<ChatResponse> chat(@RequestBody ChatRequest chatRequest) {
    String answer = chatbotService.chat(chatRequest.question());
    return ResponseEntity.ok(new ChatResponse(answer));
}

record ChatRequest(String question) {}

record ChatResponse(String answer) {}
```

We'll use the above API endpoint to interact with our chatbot later in the tutorial.

4. Creating a Custom MCP Server

In addition to using pre-built MCP servers, **we can create our own MCP servers to extend the capabilities of our chatbot with our business logic.**

Let's explore how to create a custom MCP server using Spring AI.

We'll be creating a new Spring Boot application in this section.

4.1. Dependencies (//)

First, let's include the necessary dependency in our *pom.xml* file:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server-webmvc</artifactId>
  <version>1.0.0</version>
</dependency>
```

We import Spring AI's MCP server dependency (<https://mvnrepository.com/artifact/org.springframework.ai/spring-ai-starter-mcp-server-webmvc>), which provides the necessary classes for creating a custom MCP server that supports the HTTP-based SSE transport.

4.2. Defining and Exposing Custom Tools

Next, let's define a few custom tools that our MCP server will expose.

We'll create an *AuthorRepository* class that provides methods to fetch author details:

```
class AuthorRepository {
    @Tool(description = "Get Baeldung author details using an article title")
    Author getAuthorByArticleTitle(String articleTitle) {
        return new Author("John Doe", "john.doe@baeldung.com");
    }

    @Tool(description = "Get highest rated Baeldung authors")
    List<Author> getTopAuthors() {
        return List.of(
            new Author("John Doe", "john.doe@baeldung.com"),
            new Author("Jane Doe", "jane.doe@baeldung.com")
        );
    }

    record Author(String name, String email) {
    }
}
```

For our demonstration, we're returning hardcoded author details, but in a real application, the tools would typically interact with a database or an external API.

We annotate our two methods with *@Tool* annotation and provide a brief *description* for each of them. **The *description* helps the AI model decide if and when to call the tools based on the user input and incorporate the result in its response.**

Next, let's register our author tools with the MCP server:

```
@Bean
ToolCallbackProvider authorTools() {
    return MethodToolCallbackProvider
        .builder()
        .toolObjects(new AuthorRepository())
        .build();
}
```

We use the *MethodToolCallbackProvider* to create a *ToolCallbackProvider* bean from the tools defined in our *AuthorRepository* class. **The methods annotated with *@Tool* will get exposed as MCP tools when the**

application starts up. (//)

Alternatively, **we can dynamically register tools at runtime based on specific conditions:**

```
@Bean
CommandLineRunner commandLineRunner(
    McpSyncServer mcpSyncServer,
    @Value("${com.baeldung.author-tools.enabled:false}") boolean authorToolsEnabled
) {
    return args -> {
        if (authorToolsEnabled) {
            ToolCallback[] toolCallbacks = ToolCallbacks.from(new AuthorRepository());
            List<SyncToolSpecification> tools = McpToolUtils.toSyncToolSpecifications(toolCallbacks);
            tools.forEach(tool -> {
                mcpSyncServer.addTool(tool);
                mcpSyncServer.notifyToolsListChanged();
            });
        }
    };
}
```

Here, we inject the *McpSyncServer* bean, automatically created by Spring AI, to conditionally register our tools based on a configuration property. Similar to *addTool()*, the *McpSyncServer* class also provides the *removeTool()* method to remove certain tool(s).

For our demonstration, we're using the *CommandLineRunner* (/running-setup-logic-on-startup-in-spring#6-spring-boot-commandlinrunner) interface; however, we can add/remove tools when a REST API is invoked or in response to an application event. This is particularly **useful for enabling or disabling features based on user permissions, subscription tiers, or other business logic**.

4.3. Configuring an MCP Client for Our Custom MCP Server

Finally, to use our custom MCP server in our chatbot application, we need to configure an MCP client against it:

```
spring:
  ai:
    mcp:
      client:
        sse:
          connections:
            author-tools-server:
              url: http://localhost:8081
```

In the *application.yaml* file, we configure a new client against our custom MCP server. Note that we're using the SSE transport type here.

This configuration assumes the MCP server to be running at *http://localhost:8081*. We need to make sure to update the *url* if it's running on a different host or port.

Additionally, if we've enabled our MCP server to add or remove tools at runtime dynamically, **we can register a listener to detect these tool changes:**

```
@Bean
McpSyncClientCustomizer mcpSyncClientCustomizer() {
    return (name, mcpClientSpec) -> {
        mcpClientSpec.toolsChangeConsumer(tools -> {
            logger.info("Detected tools changes.");
        });
    };
}
```

Here, we define a bean of type *McpSyncClientCustomizer* and register a listener using the *toolsChangeConsumer()* method. While we're simply logging here for simplicity, in a real application, we can refresh (/spring-reinitialize-singleton-bean) the *ChatClient* bean or programmatically restart (/java-restart-spring-boot-app) our application.

With this configuration, **our MCP client can now invoke the tools exposed by our custom server, in addition to the tools provided by the Brave Search and Filesystem MCP servers.**

5. Interacting With Our Chatbot

Now that we've built our chatbot and integrated it with various MCP servers, let's interact with it and test it out. We'll use the HTTPie (/httpie-http-client-command-line) CLI to invoke the chatbot's API endpoint:

```
http POST :8080/chat question="How much was Elon Musk's initial offer to buy OpenAI in 2025?"
```

Here, **we send a simple *question* to the chatbot about an event that occurred after the LLM's knowledge cut-off** (<https://github.com/HaoooWang/llm-knowledge-cutoff-dates?tab=readme-ov-file#:~:text=Claude%203.5%20Sonnet,Source>) date. Let's see what we get as a response:

```
{
  "answer": "Elon Musk's initial offer to buy OpenAI was $97.4 billion. [Source](https://www.reuters.com/t
```

As we can see, **the chatbot was able to perform a web search using the configured Brave Search MCP server and provide an accurate answer along with a source.**

Next, **let's verify that the chatbot can perform filesystem operations using the Filesystem MCP server:**

```
http POST :8080/chat question="Create a text file named 'mcp-demo.txt' with content 'This is awesome!'."
```

We instruct the chatbot to create an *mcp-demo.txt* file with certain content. Let's see if it's able to fulfill the request:

```
{
  "answer": "The text file named 'mcp-demo.txt' has been successfully created with the content you specified"
```

The chatbot responds with a successful response. We can verify that the file was created in the directory we specified in the *application.yaml* file.

Finally, **let's verify if the chatbot can call one of the tools exposed by our custom MCP server.** We'll enquire about the author details by mentioning an article title:


```
http POST /3050/chat question=Who wrote the article 'Testing CORS in Spring Boot?' on Baeldung, and how can
```

Let's invoke the API and see if the chatbot response contains the hardcoded author details:

```
{  
  "answer": "The article 'Testing CORS in Spring Boot' on Baeldung was written by John Doe. You can contact
```

The above response verifies that the chatbot fetches the author details using the `getAuthorByArticleTitle()` tool that our custom MCP server exposes.

We highly recommend setting up the codebase locally and playing around with the chatbot using different prompts.

6. Conclusion

In this article, we've explored the Model Context Protocol and implemented its client-server architecture using Spring AI.

First, we built a simple chatbot using Anthropic's Claude 4 Opus model to act as our MCP host.

Then, to provide our chatbot with web search capability and enable it to execute filesystem operations, we configured MCP clients against pre-built MCP server implementations of Brave Search API and Filesystem.

Finally, we created a custom MCP server and configured its corresponding MCP client inside our MCP host application.

The code backing this article is available on GitHub. Once you're **logged in as a Baeldung Pro Member (/members/)**, start learning and coding on the project.



Get started with Spring Boot and with core Spring, through the *Learn Spring* course:

>> CHECK OUT THE COURSE (/Course-LS-NPI-EA-3-bahh6)

4 COMMENTS



Oldest ▼

[View Comments](#)

COURSES

[ALL COURSES \(/COURSES/ALL-COURSES\)](/COURSES/ALL-COURSES/)

[BAELDUNG ALL ACCESS \(/COURSES/ALL-ACCESS\)](/COURSES/ALL-ACCESS/)

[BAELDUNG ALL TEAM ACCESS \(/COURSES/ALL-ACCESS-TEAM\)](/COURSES/ALL-ACCESS-TEAM/)

[LOGIN COURSE PLATFORM \(HTTPS://WWW.BAELDUNG.COM/MEMBERS/ACCOUNT\)](https://www.baeldung.com/members/account)

SERIES

[JAVA "BACK TO BASICS" TUTORIAL \(/JAVA-TUTORIAL\)](/JAVA-TUTORIAL/)

[LEARN SPRING BOOT SERIES \(/SPRING-BOOT\)](/SPRING-BOOT/)

[SPRING TUTORIAL \(/SPRING-TUTORIAL\)](/SPRING-TUTORIAL/)

[GET STARTED WITH JAVA \(/GET-STARTED-WITH-JAVA-SERIES\)](/GET-STARTED-WITH-JAVA-SERIES/)

[SPRING FRAMEWORK INTRODUCTION \(/SPRING-INTRO\)](/SPRING-INTRO/)

[ALL ABOUT STRING IN JAVA \(/JAVA-STRING\)](/JAVA-STRING/)

[SECURITY WITH SPRING \(/SECURITY-SPRING\)](/SECURITY-SPRING/)

ABOUT

[ABOUT BAELDUNG \(/ABOUT\)](/ABOUT/)

[THE FULL ARCHIVE \(/FULL_ARCHIVE\)](/FULL_ARCHIVE/)

[EDITORS \(/EDITORS\)](/EDITORS/)

[OUR PARTNERS \(/PARTNERS/\)](/PARTNERS/)

[PARTNER WITH BAELDUNG \(/PARTNERS/WORK-WITH-US\)](/PARTNERS/WORK-WITH-US/)

[EBOOKS \(/LIBRARY/\)](/LIBRARY/)

[FAQ \(/LIBRARY/FAQ\)](/LIBRARY/FAQ/)



[BAELDUNG PRO \(/MEMBERS/\)](/MEMBERS/)

[TERMS OF SERVICE \(/TERMS-OF-SERVICE\)](/TERMS-OF-SERVICE/)

[PRIVACY POLICY \(/PRIVACY-POLICY\)](/PRIVACY-POLICY/)

[COMPANY INFO \(/BAELDUNG-COMPANY-INFO\)](/BAELDUNG-COMPANY-INFO/)

[CONTACT \(/CONTACT\)](/CONTACT/)

[PRIVACY MANAGER](#)